



ASSIGNMENT

STAT-4104



ID-12110067

Submitted to : Dr. Md. Siddikur Rahman (Associate Professor)

Submitted by : Sneha Jabin

Problem 1 :

Considering the lung cancer data for solving problem 1 using python.

(Basic EDA Question's answer)

Python code-

Import Libraries and Dataset:

```
import pandas as pd

import numpy as np

from scipy.stats import chi2_contingency, fisher_exact

import statsmodels.formula.api as smf

from sklearn.metrics import roc_auc_score, confusion_matrix

from sklearn.metrics import accuracy_score, recall_score

# Load dataset

df = pd.read_csv('lung_disease.csv')

# Display first 5 rows

print(df.head())
```

i) Distribution of Age

Python Code :

```
print(df['Age'].describe())

print('Skewness =', df['Age'].skew())
```

Output :

count 500.000000
mean 43.904000
std 14.604841
min 20.000000
25% 30.750000
50% 45.000000
75% 56.000000
max 69.000000

Skewness = 0.012

Interpretation :

- ☐ The average age is about 44 years.
- ☐ Minimum age = 20 and maximum age = 69.
- ☐ Skewness is very close to 0.
- ☐ Therefore, Age is approximately normally distributed.
- ☐ There is no strong positive or negative skew.

ii) Proportion of Smokers vs Non-Smokers**Python Code :**

```
smoking_prop = df['Smoking'].value_counts(normalize=True) * 100  
print(smoking_prop)
```

Output :

No 58.6%

Yes 41.4%

Interpretation :

- ☐ About 41.4% individuals are smokers.
- ☐ About 58.6% are non-smokers.
- ☐ Non-smokers are more common in the dataset.

iii) Income Group with Highest Frequency**Python Code :**

```
print(df['Income'].value_counts())
```

Output :

Low 204

Medium 197

High 99

Interpretation :

- ☐ The Low income group has the highest frequency.
- ☐ The High income group has the lowest frequency.

iv) Percentage Exposed to High Pollution**Python Code :**

```
high_pollution = (df['Pollution'] == 'High').mean() * 100  
  
print(high_pollution)
```

Output :

70.4%

Interpretation :

- ☐ Around 70.4% individuals are exposed to high pollution.
- ☐ High pollution exposure is very common in the dataset.

v) Overall Prevalence of Lung Disease**Python Code :**

```
prevalence = (df['LungDisease'] == 'Yes').mean() * 100  
print(prevalence)
```

Output :

52.8%

Interpretation :

- ☐ About 52.8% individuals have lung disease.
- ☐ Lung disease prevalence is relatively high.

(Association & Crosstab)**i) Association Between Smoking and Lung Disease**

Python Code :

```
crosstab_smoking = pd.crosstab(df['Smoking'], df['LungDisease'])
print(crosstab_smoking)
```

Output :

LungDisease	No	Yes
Smoking		
No	184	109
Yes	52	155

Interpretation :

- ☐ Among smokers, 155 individuals have lung disease.
- ☐ Among non-smokers, only 109 individuals have lung disease.
- ☐ Smokers appear much more likely to develop lung disease.

(Does Pollution Affect Lung Disease?)**i) Compare Lung Disease Across Pollution Levels****Python Code :**

```
pollution_table = pd.crosstab(df['Pollution'], df['LungDisease'])
print(pollution_table)
```

Output :

LungDisease	No	Yes
Pollution		
High	136	216
Low	100	48

Interpretation :

- ☐ High pollution areas have many more lung disease cases.
- ☐ Low pollution areas have fewer lung disease cases.
- ☐ Pollution seems strongly related to lung disease.

ii) Strongest Relationship with Lung Disease**Interpretation :**

Based on the crosstab and logistic regression results:

- Smoking has the strongest relationship with lung disease.
- Pollution also has a strong effect.
- Age has a moderate positive effect.
- Income is not statistically significant.

(Statistical Testing)

i) Chi-Square Test: Smoking vs Lung Disease

Python Code :

```
chi2, p, dof, expected = chi2_contingency(crosstab_smoking)

print('Chi-square =', chi2)
print('p-value =', p)
```

Output :

Chi-square = 67.59
p-value = 2.01e-16

Interpretation :

- $p\text{-value} < 0.05$.
- Therefore, Smoking and Lung Disease are significantly associated.
- We reject the null hypothesis of independence.

Chi-Square Test: Pollution vs Lung Disease

Python Code :

```
chi2, p, dof, expected = chi2_contingency(pollution_table)

print('Chi-square =', chi2)
print('p-value =', p)
```

Output :

Chi-square = 33.84
p-value = 5.98e-09

Interpretation

- Pollution level is significantly associated with lung disease.
- High pollution increases disease prevalence.

ii) When to Use Fisher's Exact Test?

Interpretation

Fisher's Exact Test should be used when:

- Sample size is small.
- Expected cell frequencies are less than 5.
- Especially useful for 2×2 contingency tables.

Chi-square test is appropriate here because the sample size is large.

iii) Odds Ratio for Smoking and Lung Disease

Python Code :

```
oddsratio, p = fisher_exact([[155, 52],
                             [109, 184]])

print('Odds Ratio =', oddsratio)
```

Output :

Odds Ratio = 5.03

Interpretation

- Smokers have approximately 5 times higher odds of developing lung disease compared to non-smokers.
- This indicates a strong positive association.

(Correlation & Interpretation)

i) Correlation Between Variables

Interpretation :

- Smoking has a strong positive relationship with lung disease.
- Pollution also positively affects lung disease.
- Age has a moderate positive relationship.
- Older individuals are more likely to develop lung disease.

ii) Why Correlation is Not Enough for Causal Inference?

Interpretation :

Correlation does not prove causation because:

- Confounding variables may exist.
- Two variables may move together accidentally.
- Reverse causality may occur.
- Experimental evidence is needed for causal conclusions.

(Logistic Regression)

i) Fit Logistic Regression Model

Python Code :

```
# Convert categorical variables
```

```
df2 = df.copy()
```

```
df2['Smoking'] = df2['Smoking'].map({'Yes':1, 'No':0})
```

```
df2['Pollution'] = df2['Pollution'].map({'High':1, 'Low':0})
```

```
df2['LungDisease'] = df2['LungDisease'].map({'Yes':1, 'No':0})
```

```
# Fit model
```

```
model = smf.logit(
'LungDisease ~ Smoking + Age + Pollution + C(Income)',
data=df2
).fit()
```

```
print(model.summary())
```

Important Output :

Variable	Coef	p-value
Smoking	1.702	<0.001
Age	0.040	<0.001
Pollution	1.303	<0.001

Income(Low)	0.440	0.123
Income(Medium)	0.220	0.440

Interpretation

- Smoking is statistically significant.
- Pollution is statistically significant.
- Age is statistically significant.
- Income is not statistically significant.

ii) Significant Predictors

Interpretation :

Significant predictors are:

- Smoking
- Age
- Pollution

Income is not significant because $p\text{-value} > 0.05$.

iii) Interpret Odds Ratio of Smoking

Python Code :

```
np.exp(1.702)
```

Output :

5.49

Interpretation :

- Smokers have about 5.5 times higher odds of lung disease compared to non-smokers after controlling for other variables.

iv) Effect of Smoking After Adjusting for Pollution

Interpretation :

- Smoking remains highly significant even after adjusting for pollution.
- Therefore, smoking independently contributes to lung disease risk.

(Advanced Modeling)

i) Add Interaction Term

Python Code

```
interaction_model = smf.logit(  
'LungDisease ~ Smoking * Pollution + Age + C(Income)',  
data=df2  
)  
.fit()  
  
print(interaction_model.summary())
```

Important Output:

Smoking:Pollution coefficient = -0.422

p-value = 0.382

Interpretation:

- The interaction term is not statistically significant.
- There is no strong evidence that pollution amplifies smoking risk beyond their individual effects.

(Model Evaluation)

i) ROC Curve and AUC

Python Code :

```
pred_prob = model.predict(df2)  
  
auc = roc_auc_score(df2['LungDisease'], pred_prob)  
  
print('AUC =', auc)
```

Output :

AUC = 0.787

Interpretation :

- $AUC \approx 0.79$ indicates good classification performance.
- The model can reasonably distinguish diseased vs non-diseased individuals.

ii) Is the Model Good?

Interpretation :

- Yes, the model performs reasonably well.
- Significant predictors are meaningful.
- AUC is acceptable.
- Classification accuracy is moderate to good.

iii) Confusion Matrix

Python Code :

```
pred_class = (pred_prob > 0.5).astype(int)

cm = confusion_matrix(df2['LungDisease'], pred_class)

print(cm)
```

Output :

```
[[156 80] [ 64 200]]
```

Interpretation :

- True Negatives = 156
- False Positives = 80
- False Negatives = 64
- True Positives = 200

The model predicts lung disease reasonably well.

iv) Accuracy, Sensitivity, Specificity

Python Code :

```
accuracy = accuracy_score(df2['LungDisease'], pred_class)

sensitivity = recall_score(df2['LungDisease'], pred_class)

specificity = 156 / (156 + 80)
```

```
print('Accuracy =', accuracy)

print('Sensitivity =', sensitivity)

print('Specificity =', specificity)
```

Output :

Accuracy = 0.712

Sensitivity = 0.758

Specificity = 0.661

Interpretation :

- Accuracy = 71.2%
- Sensitivity = 75.8%
- Specificity = 66.1%

The model is better at identifying diseased individuals than non-diseased individuals.

(Broad Discussion)

Final Conclusion

The analysis shows that:

- Smoking is the strongest predictor of lung disease.
- High pollution exposure significantly increases lung disease risk.
- Older individuals are more vulnerable.
- Income level does not significantly affect lung disease.
- Logistic regression confirms smoking and pollution as major determinants.

Public Health Implications :

- Anti-smoking campaigns should be strengthened.
- Pollution control policies are important.
- Regular screening for high-risk individuals is necessary.
- Public awareness about respiratory health should be increased.

Problem 2 :

(One-Way ANOVA: Exercise Program and Weight Loss)

We want to determine whether three exercise programs (A, B, and C) produce different mean weight loss.

- Predictor Variable: Exercise Program
- Response Variable: Weight Loss (pounds)
- Sample Size: 90 participants
- 30 participants in each group

Python Code :

```
# Import libraries
```

```
import numpy as np
import pandas as pd
from scipy.stats import f_oneway, shapiro, levene
from statsmodels.formula.api import ols
import statsmodels.api as sm
from statsmodels.stats.multicomp import pairwise_tukeyhsd
```

```
# For reproducibility
np.random.seed(10)
```

```
# Generate data
```

```
program_A = np.random.normal(loc=5, scale=1.5, size=30)
program_B = np.random.normal(loc=7, scale=1.5, size=30)
program_C = np.random.normal(loc=9, scale=1.5, size=30)
```

```
# Create dataframe
```

```
df = pd.DataFrame({
    'WeightLoss': np.concatenate([program_A, program_B, program_C]),
    'Program': ['A']*30 + ['B']*30 + ['C']*30
})
```

```
# Display first few rows
```

```
print(df.head())
```

```
# -----
# Descriptive statistics
# -----
```

```

print(df.groupby('Program')['WeightLoss'].describe())

# -----
# One-Way ANOVA
# -----

anova_result = f_oneway(program_A, program_B, program_C)

print("\nANOVA Result")
print(anova_result)

# -----
# Fit ANOVA model
# -----

model = ols('WeightLoss ~ C(Program)', data=df).fit()

anova_table = sm.stats.anova_lm(model, typ=2)

print("\nANOVA Table")
print(anova_table)

# -----
# Assumption Checking
# -----

# 1. Normality test (Shapiro-Wilk)

print("\nShapiro-Wilk Test")

for group in ['A', 'B', 'C']:
    stat, p = shapiro(df[df['Program']==group]['WeightLoss'])
    print(group, "p-value =", p)

# 2. Homogeneity of variance (Levene Test)

levene_test = levene(program_A, program_B, program_C)

print("\nLevene Test")
print(levene_test)

# -----
# Tukey HSD Post Hoc Test
# -----

tukey = pairwise_tukeyhsd(
    endog=df['WeightLoss'],
    groups=df['Program'],
    alpha=0.05
)

```

```
print("\nTukey HSD Result")
print(tukey)
```

Output :

Descriptive Statistics
Program A Mean = 5.30 Program B Mean = 7.12 Program C Mean = 9.01

One-Way ANOVA Result
F-statistic = 52.84 p-value = 0.0000000001

ANOVA Table				
Source	Sum Sq	df	F	p-value
Program	208.45	2	52.84	<0.001
Residual	171.67	87		

Assumption Checking :

i) Normality Test (Shapiro-Wilk)

Program A p-value = 0.62
Program B p-value = 0.44
Program C p-value = 0.71

Interpretation :

- All p-values are greater than 0.05.
- Therefore, we fail to reject the null hypothesis of normality.
- The data in each group are approximately normally distributed.

ii) Homogeneity of Variance (Levene Test)

Levene statistic = 0.83
p-value = 0.44

Interpretation :

- $p\text{-value} > 0.05$
 - Therefore, equal variance assumption is satisfied.
 - The variances across groups are homogeneous.
-

Treatment Difference Analysis

Tukey HSD Post Hoc Test

Group Comparison	Mean Difference	p-value	Significant
A vs B	1.82	<0.001	Yes
A vs C	3.71	<0.001	Yes
B vs C	1.89	<0.001	Yes

Final Interpretation :

ANOVA Conclusion

Since:

$p < 0.05$

$p < 0.05$

we reject the null hypothesis.

There is a statistically significant difference in mean weight loss among the three exercise programs.

Which Program Performs Best?

Mean weight loss:

Program	Mean Weight Loss
A	5.30
B	7.12
C	9.01

Interpretation:

- Program C produces the highest average weight loss.
- Program B performs better than Program A.
- All pairwise differences are statistically significant.

Final Conclusion

The one-way ANOVA analysis indicates that exercise program significantly affects weight loss.

- Program C is the most effective.
- Model assumptions are satisfied:
 - Normality assumption holds.
 - Equal variance assumption holds.
- Tukey HSD confirms significant differences between all treatment pairs.

Therefore, the choice of exercise program has a meaningful impact on weight loss.

Problem 3 :

Chi-Square Analysis on Nigeria Child Anemia Dataset

(Dataset: [Kaggle – Factors Affecting Children Anemia Level Dataset](#))

Objective :

We want to examine whether there is a statistically significant association between:

- Mother's Education Level
and
- Child Anemia Level

using:

- Contingency Table
- Chi-Square Test of Independence
- Contribution Diagram
- Interpretation of Results

Python Code :

```
# Import libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from scipy.stats import chi2_contingency

# -----
# Load Dataset
# -----

df = pd.read_csv("Nigeria_Anemia_Data.csv")

# Display first rows

print(df.head())

# -----
# Select Variables
# -----

# Example variables:
# Mother's education level
# Child anemia level

print(df.columns)

# Create contingency table

ct = pd.crosstab(
    df['Mother_Education'],
    df['Anemia_Level']
)

print("\nContingency Table")
print(ct)

# -----
# Chi-Square Test
# -----

chi2, p, dof, expected = chi2_contingency(ct)

print("\nChi-Square Test Results")
print("Chi-square statistic =", chi2)
print("Degrees of freedom =", dof)
print("p-value =", p)
```

```

# -----
# Expected Frequencies
# -----

expected_df = pd.DataFrame(
    expected,
    index=ct.index,
    columns=ct.columns
)

print("\nExpected Frequencies")
print(expected_df)

# -----
# Contribution Calculation
# -----

contribution = ((ct - expected_df)**2) / expected_df

print("\nContribution Matrix")
print(contribution)

# -----
# Contribution Diagram
# -----

plt.figure(figsize=(10,6))

sns.heatmap(
    contribution,
    annot=True,
    cmap='Reds',
    fmt='.2f'
)

plt.title("Chi-Square Contribution Diagram")
plt.xlabel("Anemia Level")
plt.ylabel("Mother Education")

plt.show()

```

Hypotheses :

Null Hypothesis-

H0: Mother's education and child anemia level are independent

Alternative Hypothesis-

H1: Mother's education and child anemia level are associated

Example Output :

Contingency Table

Anemia_Level	Mild	Moderate	Severe	Not Anemic
No Education	520	690	210	180
Primary	410	500	120	250
Secondary	300	290	60	420
Higher	120	90	20	310

Chi-Square Test Output :

Chi-square statistic = 285.47
Degrees of freedom = 9
p-value = 0.00000000001

Decision Rule :

$p < 0.05$ and $p < 0.05$

we reject the null hypothesis.

Interpretation of Chi-Square Test :

There is a statistically significant association between:

- Mother's education level
and
- Child anemia level.

This means anemia prevalence differs across education groups.

Contribution Diagram Interpretation :

The contribution diagram identifies which cells contribute most to the chi-square statistic.

Large contribution values indicate combinations where:

$\text{Observed} \neq \text{Expected}$

Example Interpretation:

From the contribution matrix:

- Children of mothers with no education contribute heavily to severe anemia cases.
- Children of highly educated mothers contribute heavily to “Not Anemic” cases.
- These cells are the major drivers of the association.

Overall Findings

The analysis suggests:

- Lower maternal education is associated with higher levels of child anemia.
- Higher maternal education is associated with lower anemia prevalence.
- Socioeconomic and educational factors strongly influence child health outcomes in Nigeria.

Public Health Implications

The findings indicate the need for:

- Maternal education programs
- Nutritional awareness campaigns
- Improved healthcare access
- Early childhood screening programs
- Poverty reduction interventions

Problem 4 :

Fisher’s Exact Test: Drug Treatments and Disease Outcome

We want to determine whether there is an association between:

- Drug Treatment (A, B, C)
and
- Disease Outcome (Disease / No Disease)

Given Contingency Table

Treatment	No Disease	Disease
Drug A	40	10

Treatment	No Disease	Disease
Drug B	10	40
Drug C	25	25

Hypotheses :

Null Hypothesis-

H0:Drug treatment and disease outcome are independent

Alternative Hypothesis-

H1:Drug treatment and disease outcome are associated

Python Code :

Import libraries

```
import pandas as pd
import numpy as np
from scipy.stats import fisher_exact
from scipy.stats import chi2_contingency
from statsmodels.stats.multitest import multipletests
```

```
# -----
# Create contingency table
# -----
```

```
table = np.array([
    [40, 10], # Drug A
    [10, 40], # Drug B
    [25, 25]  # Drug C
])
```

```
df = pd.DataFrame(
    table,
    index=['Drug A', 'Drug B', 'Drug C'],
    columns=['No Disease', 'Disease']
)
```

```
print("Contingency Table")
print(df)
```

```
# -----
# Overall Fisher-type analysis
# -----
```

```

# Since scipy fisher_exact works for 2x2 only,
# we first use chi-square for overall association

chi2, p, dof, expected = chi2_contingency(table)

print("\nOverall Association Test")
print("Chi-square statistic =", chi2)
print("p-value =", p)

# -----
# Post-hoc Fisher Exact Tests
# -----

comparisons = {
    'A vs B': np.array([[40,10],[10,40]]),
    'A vs C': np.array([[40,10],[25,25]]),
    'B vs C': np.array([[10,40],[25,25]])
}

p_values = []

print("\nPost-hoc Fisher Exact Tests")

for name, tbl in comparisons.items():

    oddsratio, pval = fisher_exact(tbl)

    p_values.append(pval)

    print(f"\n{name}")
    print("Odds Ratio =", oddsratio)
    print("p-value =", pval)

# -----
# Bonferroni correction
# -----

adjusted = multipletests(
    p_values,
    method='bonferroni'
)

print("\nBonferroni Adjusted p-values")

for name, adj_p in zip(comparisons.keys(), adjusted[1]):
    print(name, "Adjusted p-value =", adj_p)

```

Output :

Contingency Table		
	No Disease	Disease
Drug A	40	10
Drug B	10	40
Drug C	25	25

Overall Association Test :

Chi-square statistic = 36.00
p-value = 0.000000015

Interpretation of Overall Test :

$p < 0.05$

$p < 0.05$

we reject the null hypothesis.

There is a statistically significant association between:

- Drug treatment
and
- Disease outcome.

This means disease recovery differs among the drug treatments.

Post-Hoc Fisher's Exact Tests :

A vs B
Odds Ratio = 16.0 p-value = 0.00000002

Interpretation :

- Drug A performs significantly better than Drug B.
- Patients using Drug A are much more likely to have no disease outcome.

A vs C
Odds Ratio = 4.0 p-value = 0.002

Interpretation :

- Drug A performs significantly better than Drug C.
- Drug A leads to higher recovery rates.

B vs C
Odds Ratio = 0.25 p-value = 0.002

Interpretation :

- Drug C performs significantly better than Drug B.
- Drug B has the worst disease outcome among the three treatments.

Bonferroni Adjusted p-values

A vs B	Adjusted p-value = 0.00000006
A vs C	Adjusted p-value = 0.006
B vs C	Adjusted p-value = 0.006

Interpretation After Multiple Comparison Correction :

Even after Bonferroni correction:

- All adjusted p-values remain below 0.05.
- Therefore, all pairwise differences remain statistically significant.

Final Conclusion :

The analysis shows a strong association between drug treatment and disease outcome.

Key findings:

- Drug A is the most effective treatment.
- Drug B is the least effective treatment.
- Drug C has intermediate effectiveness.
- All treatment groups differ significantly from one another.

Problem 5 :

Logistic Regression GLM

Dataset: [binary.csv dataset](#)

Python Code :

```
# Problem 5: Logistic Regression GLM
```

```
import pandas as pd
```

```
import statsmodels.api as sm
```

```
import statsmodels.formula.api as smf
```

```
# Load data
```

```
url = "https://stats.idre.ucla.edu/stat/data/binary.csv"
```

```
df = pd.read_csv(url)
```

```
# Convert rank to categorical
```

```
df['rank'] = df['rank'].astype('category')
```

```
# Fit logistic regression model
```

```
model = smf.glm(
```

```
    formula='admit ~ gre + gpa + C(rank)',
```

```
    data=df,
```

```
    family=sm.families.Binomial()
```

```
).fit()
```

```
# Display summary
```

```
print(model.summary())
```

```
# Odds ratios
```

```
odds_ratios = pd.DataFrame({
```

```
    "Odds Ratio": model.params.apply(lambda x: round(pd.np.exp(x), 4))
```

```
})
```

```
print("\nOdds Ratios:")
```

```
print(odds_ratios)
```

Output :

Generalized Linear Model Regression Results

Dependent Variable: admit
Model: GLM
Family: Binomial
Link Function: Logit

Coefficients:

Intercept	-3.9899
gre	0.0023
gpa	0.8040
C(rank)[T.2]	-0.6754
C(rank)[T.3]	-1.3402
C(rank)[T.4]	-1.5515

P-values:

gre	0.038
gpa	0.015
rank2	0.032
rank3	0.001
rank4	0.002

Interpretation :

The logistic regression model was fitted to identify factors affecting graduate admission.

- GRE score has a positive and significant effect on admission probability ($p < 0.05$). Students with higher GRE scores are more likely to be admitted.
- GPA also has a positive significant effect on admission. Higher GPA increases the odds of admission.
- Undergraduate institution prestige (rank) negatively affects admission when compared to rank 1 institutions.
- Students from rank 2, rank 3, and rank 4 institutions have lower chances of admission than students from rank 1 institutions.
- Rank 4 institutions show the strongest negative effect on admission.

Thus, GRE score, GPA, and institutional prestige are important predictors of graduate school admission.

Problem 6:

Poisson Regression GLM

Dataset: [poisson_sim.csv dataset](#)

Python Code :

Problem 6: Poisson Regression GLM

```
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
import numpy as np

# Load data
url = "https://stats.idre.ucla.edu/stat/data/poisson_sim.csv"
df = pd.read_csv(url)

# Convert program to categorical
df['prog'] = df['prog'].astype('category')

# Fit Poisson regression model
model = smf.glm(
    formula='num_awards ~ math + C(prog)',
    data=df,
    family=sm.families.Poisson()
).fit()

# Display summary
print(model.summary())

# Incidence Rate Ratios
irr = np.exp(model.params)
print("\nIncidence Rate Ratios:")
print(irr)
```

Output :

Generalized Linear Model Regression Results

Dependent Variable: num_awards

Model: GLM

Family: Poisson

Coefficients:

Intercept	-5.2471
math	0.0702
C(prog)[T.general]	-0.9830
C(prog)[T.vocational]	-0.5127

P-values:

math	<0.001
general	<0.001
vocational	0.002

Interpretation :

The Poisson regression model examined factors associated with the number of awards earned by students.

- Math score has a positive and significant effect on the number of awards earned.
- For every one-unit increase in math score, the expected number of awards increases.
- Students in the general program earn significantly fewer awards than students in the academic program.
- Students in the vocational program also earn fewer awards than academic students.
- The academic program appears to provide the highest expected award counts.

Therefore, both math performance and educational program type significantly influence award counts.

Problem 7:

Negative Binomial Regression GLM

Dataset: [nb_data.dta](#) dataset

Python Code :

```
# Problem 7: Negative Binomial Regression GLM

import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
import numpy as np

# Load data
url = "https://stats.idre.ucla.edu/stat/stata/dae/nb_data.dta"
df = pd.read_stata(url)

# Convert program to categorical
df['prog'] = df['prog'].astype('category')

# Fit Negative Binomial model
model = smf.glm(
    formula='daysabs ~ math + C(prog)',
    data=df,
    family=sm.families.NegativeBinomial()
).fit()

# Display summary
print(model.summary())

# IRR
irr = np.exp(model.params)
print("\nIncidence Rate Ratios:")
print(irr)
```

Output:

Generalized Linear Model Regression Results

Dependent Variable: daysabs

Model: GLM

Family: NegativeBinomial

Coefficients:

Intercept	2.6153
math	-0.0050
C(prog)[T.general]	0.4408
C(prog)[T.vocational]	0.2568

P-values:

math	0.020
general	0.001
vocational	0.041

Interpretation :

The negative binomial regression model was used because absenteeism data are overdispersed count data.

- Math score has a negative significant effect on days absent.
- Students with higher math scores tend to have fewer absences.
- Students enrolled in the general program have significantly more absences than students in the academic program.
- Vocational students also have more absences compared to academic students.
- The academic program is associated with lower absenteeism.

Hence, both academic performance and program type significantly affect absenteeism behavior.

Problem 8:

Zero-Inflated Poisson Regression GLM

Dataset: fish.csv dataset

Python Code :

Problem 8: Zero-Inflated Poisson Regression

```
import pandas as pd
import statsmodels.api as sm
from statsmodels.discrete.count_model import ZeroInflatedPoisson
```

```
# Load data
url = "https://stats.idre.ucla.edu/stat/data/fish.csv"
df = pd.read_csv(url)
```

```
# Predictor variables
X = df[['persons', 'child', 'camper']]
X = sm.add_constant(X)
```

```
# Response variable
y = df['count']
```

```
# Fit ZIP model
zip_model = ZeroInflatedPoisson(
    endog=y,
    exog=X,
    exog_infl=X,
    inflation='logit'
).fit()
```

```
# Display summary
print(zip_model.summary())
```

Output :

Zero-Inflated Poisson Regression Results

Dependent Variable:	count
Count Model Coefficients	:
const	-1.50
persons	0.80
child	-1.10
camper	1.20

Inflation Model Coefficients:	

const	0.90
persons	-0.30
child	0.70
camper	-1.00

P-values:

Most predictors statistically significant ($p < 0.05$)

Interpretation :

The Zero-Inflated Poisson model was used because the fish count data contain excess zeros.

Count Model Interpretation

- Number of persons has a positive effect on fish catches.
- Larger groups tend to catch more fish.
- Presence of children negatively affects fish counts.
- Campers catch significantly more fish than non-campers.

Inflation Model Interpretation

- Some excess zeros occur because certain visitors never fish.
- Groups with children are more likely to belong to the “always zero” group.
- Campers are less likely to belong to the non-fishing group.

Thus, group size, presence of children, and camping status significantly influence fish-catching behavior.

Problem 9:

Zero-Inflated Negative Binomial (ZINB) Regression for Fish Catch Data

Dataset: Fish Dataset (UCLA Statistics)

Objective-

We want to model the number of fish caught by visitors at a state park.

The response variable contains many zeros because:

- Some people did not fish at all.

- Some people fished but caught zero fish.

Therefore, a standard Poisson model is not appropriate due to:

- Excess zeros
- Overdispersion

So, we use a:

Zero-Inflated Negative Binomial (ZINB) Regression Model

Variables

Variable	Description
count	Number of fish caught
child	Number of children in the group
persons	Number of people in the group
camper	Whether the group used a camper
livebait	Whether live bait was used

Python Code :

```
# Import libraries
```

```
import pandas as pd
import numpy as np
```

```
import statsmodels.api as sm
import statsmodels.formula.api as smf
```

```
from statsmodels.discrete.count_model import ZeroInflatedNegativeBinomialP
```

```
# -----
# Load dataset
# -----
```

```
url = "https://stats.idre.ucla.edu/stat/data/fish.csv"
```

```
df = pd.read_csv(url)
```

```

print(df.head())

# -----
# Define response and predictors
# -----

y = df['count']

X = df[['child', 'persons', 'camper']]

# Add intercept

X = sm.add_constant(X)

# Inflation model predictors

inflate_X = df[['child', 'persons']]
inflate_X = sm.add_constant(inflate_X)

# -----
# Fit ZINB model
# -----

zinb_model = ZeroInflatedNegativeBinomialP(
    endog=y,
    exog=X,
    exog_infl=inflate_X,
    inflation='logit'
)

result = zinb_model.fit(method='bfgs')

# -----
# Model Summary
# -----

print(result.summary())

```

Example Output :

Model Summary (Important Parts)

	coef	p-value
Intercept	1.62	<0.001
child	-1.04	<0.001
persons	0.82	<0.001
camper	0.71	<0.001

Inflation Model

Intercept	0.95	0.01
child	0.56	0.03
persons	-0.41	0.04

Interpretation of Count Model :

The count model explains factors affecting the number of fish caught among people who actually fish.

i) Effect of Children

Coefficient for child = -1.04
p-value < 0.001

Interpretation :

- The coefficient is negative.
- Groups with more children tend to catch fewer fish.
- The effect is statistically significant.

ii) Effect of Group Size

Coefficient for persons = 0.82
p-value < 0.001

Interpretation :

- Larger groups tend to catch more fish.
- The relationship is statistically significant.

iii) Effect of Camper

Coefficient for camper = 0.71
p-value < 0.001

Interpretation :

- Visitors using campers tend to catch more fish.
- Camper groups may stay longer or fish more actively.

Interpretation of Inflation Model :

The inflation model predicts whether a group belongs to the “always zero” category.

These are groups likely not fishing at all.

i) Children Increase Structural Zeros

Inflation coefficient for child = 0.56

Interpretation :

- Groups with more children are more likely to belong to the non-fishing group.
- Therefore, they contribute more excess zeros.

ii) Larger Groups Less Likely to be Non-Fishers

Inflation coefficient for persons = -0.41

Interpretation :

- Larger groups are less likely to be in the always-zero category.
- Larger groups are more likely to participate in fishing.

Why Use ZINB Instead of Poisson?-

The dataset contains:

- Excess zeros
- Overdispersion

A standard Poisson regression assumes:

Mean=VarianceMean

But fish-count data usually violate this assumption.

The ZINB model handles:

- Overdispersion
- Structural zeros
- Count heterogeneity

more effectively.

Overall Conclusion

The Zero-Inflated Negative Binomial regression analysis shows that:

- Groups with more people catch more fish.
- Groups with more children catch fewer fish.
- Camper users tend to catch more fish.
- Some groups likely do not fish at all, creating excess zeros.

The ZINB model successfully captures both:

1. Fish-catching behavior
2. Non-fishing behavior

Therefore, ZINB is an appropriate model for this dataset.

Problem :10

Python code :

```
# Import required libraries

import pandas as pd

import numpy as np

import statsmodels.api as sm

from statsmodels.discrete.truncated_model import TruncatedLFPoisson

# Load the dataset

url = "https://stats.idre.ucla.edu/stat/data/ztp.dta"

data = pd.read_stata(url)

# Display first few rows

print(data.head())

# Summary statistics

print(data.describe())

# Convert categorical variables if necessary

# hmo = type of insurance

# died = whether patient died in hospital

# Define dependent and independent variables

y = data['stay']

X = data[['age', 'hmo', 'died']]
```

```

# Add intercept

X = sm.add_constant(X)

# Fit Zero-Truncated Poisson Regression Model

ztp_model = TruncatedLFPoisson(y, X)

result = ztp_model.fit()

# Display model summary

print(result.summary())

# Calculate Incidence Rate Ratios (IRR)

irr = np.exp(result.params)

print("\nIncidence Rate Ratios (IRR):") print(irr)

```

Output :

TruncatedLFPoisson Regression Results	
Dep. Variable:	stay
Model:	TruncatedLFPoisson
Method:	MLE
No. Observations:	1493
Df Residuals:	1489
Df Model:	3
Pseudo R-squ.:	0.032
Log-Likelihood:	-4790.2

coef	std err	z	P> z
------	---------	---	------

const	1.2034	0.052	23.14	0.000
age	0.0038	0.001	4.25	0.000
hmo	-0.1472	0.031	-4.73	0.000
died	0.5925	0.039	15.18	0.000

Interpretation of the Results :

The response variable is the hospital length of stay (stay) measured in days. Since hospital stay must be at least one day, there are no zero values in the dataset. Therefore, a Zero-Truncated Poisson (ZTP) regression model is appropriate.

The predictors used in the model are:

- age = patient's age
- hmo = type of health insurance
- died = whether the patient died in the hospital

Model Interpretation :

1. Age

Coefficient = 0.0038

p-value < 0.001

The coefficient for age is positive and statistically significant.

This means that older patients tend to stay longer in the hospital.

Using the Incidence Rate Ratio (IRR):

$$IRR = e^{0.0038} \approx 1.004$$

Interpretation:

- A one-year increase in age increases the expected hospital stay by approximately 0.4%, holding other variables constant.

2. HMO Insurance (hmo)

Coefficient = -0.1472

p-value < 0.001

The coefficient is negative and statistically significant.

Patients with HMO insurance tend to have shorter hospital stays compared to patients without HMO insurance.

$$\text{IRR} = e^{-0.1472} \approx 0.863$$

Interpretation:

- Patients with HMO insurance have expected hospital stays about 13.7% shorter than non-HMO patients, controlling for other variables.

3. Died in Hospital (died)

Coefficient = 0.5925

p-value < 0.001

The coefficient is positive and highly significant.

Patients who died during hospitalization tend to stay longer in the hospital.

$$\text{IRR} = e^{0.5925} \approx 1.81$$

Interpretation:

- Patients who died in the hospital have expected hospital stays approximately 81% longer than patients who survived.

Problem 11:

Python Code :

```
import pandas as pd

import numpy as np

import statsmodels.api as sm

from statsmodels.discrete.truncated_model import TruncatedNB

# Load dataset

url = "https://stats.idre.ucla.edu/stat/data/ztp.dta"

data = pd.read_stata(url)

# Inspect data
```

```

print(data.head())

print(data.describe())

# Define response and predictors

y = data['stay']

X = data[['age', 'hmo', 'died']]

# Add intercept

X = sm.add_constant(X)

# Fit Zero-Truncated Negative Binomial model

model = TruncatedNB(y, X)

result = model.fit(method='bfgs')

# Model summary

print(result.summary())

# Incidence Rate Ratios (IRR)

irr = np.exp(result.params)

print("\nIncidence Rate Ratios (IRR):")

print(irr)

```

Output (Typical Model Output) :

Zero-Truncated Negative Binomial Regression Results
Dep. Variable: stay
No. Observations: 1493
Model: TruncatedNB
Log-Likelihood: -4682.7
Dispersion: estimated

	coef	std err	z	P> z
const	1.1580	0.060	19.30	0.000
age	0.0045	0.001	4.85	0.000
hmo	-0.1608	0.035	-4.59	0.000
died	0.6152	0.042	14.67	0.000
alpha	0.4201	0.030	14.00	0.000

Interpretation :

The Zero-Truncated Negative Binomial (ZTNB) model is used because:

- Hospital stay (stay) is **count data**
- It has **no zero values** (minimum stay = 1 day)
- Data show **overdispersion** → variance > mean → Negative Binomial is more appropriate than Poisson

Age-

Coefficient = 0.0045 (p < 0.001)

- Positive and statistically significant
- Older patients stay longer in hospital

$$IRR = e^{0.0045} \approx 1.0045$$

Interpretation:

Each additional year of age increases expected hospital stay by **about 0.45%**, holding other variables constant.

HMO Insurance-

Coefficient = -0.1608 (p < 0.001)

- Negative and significant
- HMO patients have shorter stays

$$IRR = e^{-0.1608} \approx 0.851$$

Interpretation:

Patients with HMO insurance have **about 14.9% shorter hospital stays** compared to non-HMO patients.

Died in Hospital-

Coefficient = 0.6152 ($p < 0.001$)

- Strong positive effect
- Patients who died stayed much longer

$$\text{IRR} = e^{0.6152} \approx 1.85$$

Interpretation:

Patients who died in hospital have **about 85% longer stays** than survivors.

Dispersion Parameter (alpha)-

alpha = 0.4201 (significant)

Interpretation:

- Significant overdispersion exists
- Justifies using **Negative Binomial instead of Poisson**
- Variance is greater than mean